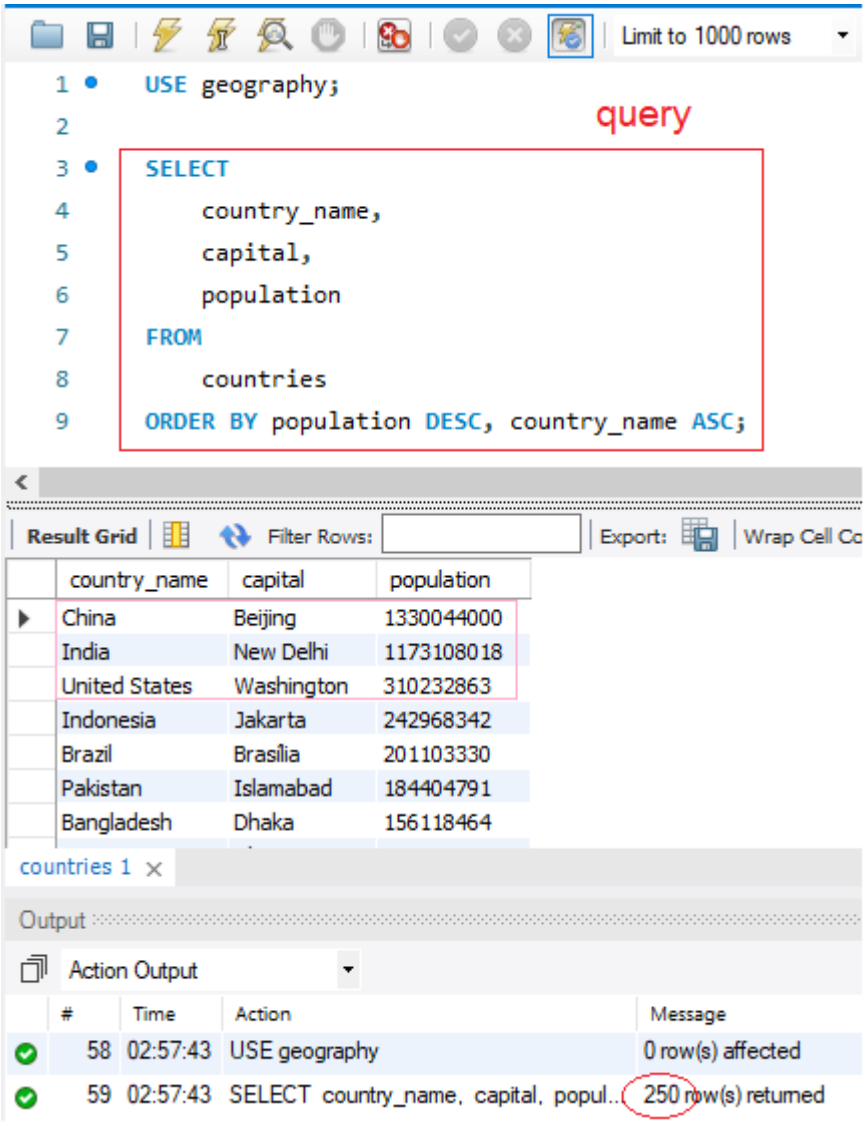
MySQL процедури

Когато искаме да запазим често повтаряща се заявка, я съхраняваме като процедура.

Например, имаме заявка:



Тази заявка не се запазва никъде. За да я съхраним, по същия начин, както имаме CREATE TABLE и CREATE VIEW, имаме и CREATE PROCEDURE.
Желателно е да уточняваме изгледите VIEWS като името им започва с v_, докато процедурите е желателно да започват с Get и задължително завършват с () – те показват, че могат да се подават входни данни и връщат изходи, по аналогичен начин както в C#.

Заявката е оградена от BEGIN – END и ограничена с DELIMITER $$ - това указва разделител, както показва името. Стандартният разделител в SQL е **;** но при процедурите указваме използване на други разделители за по-голямо разграничение между отделните вложени заявки вътре в самата процедура – там се използват ; но за разграничаване между отделните процедури, използваме друг разделител.

Могат да се използват други разделители като DELIMETER ## с един или няколко повтарящи се символа, но като цяло $$ се е наложил като най-практичен, т.к. програмистите обикновено боравят в работата си с няколко езика и в програмирането // указват коментар. Така се избягват грешки от смесване на различните езици. Освен това, трябва и да се избягва обратна наклонена черта \ , т.к. тя указва изключване на знак, например

```
SELECT 'It\'s a sunny day';
```

за да може да се приеме символа след него, а да не се прекъсне стринга.

```
1   DELIMITER $$
2
3   SELECT * FROM mountains $$
4
5   SELECT * FROM rivers $$
6         вместо ;
7   DELIMITER ##
8
9   SELECT * FROM peaks ##
10
11  SELECT * FROM continents ##
12
13  DELIMITER ; отново ;
```

Това е валиден код, който връща резултати.

Не е желателно да използваме // и \ , макар, че не връща грешки при тях – те се използват като коментари в други програмни езици като C# или за изключване на стоящият след \ знак.

```
DELIMITER //

SELECT * FROM mountains //

SELECT * FROM rivers //

DELIMITER \

SELECT * FROM peaks \

SELECT * FROM continents \

DELIMITER ;
```

Когато искаме вече кодът да се върне към нормалния си синтаксис с обикновения разделител ; указваме DELIMETER ;

```
1   USE geography;
2
3   DELIMITER $$
4
5   CREATE PROCEDURE GetCountries()
6   BEGIN
15  DELIMITER ;
```

създаване на нов разделител

$$

$$

$$

;

;

Заявката продължава по стандартния начин с ;. Ако изтрия интервала, хвърля грешка:

```
3   DELIMITER $$
4
5   CREATE PROCEDURE GetCountries()
6   BEGIN
15  DELIMITER;
```

Процедурата е адресирана до локалния сървър и целта със съхраняването е да намалят мрежовия трафик от компилиране на повтарящ се код. Вместо всеки път да се подава към сървъра един и същи код, след като веднъж той е компилиран, се съхранява в кеш паметта (вградена бързодействаща памет) и когато потрябва, просто се извиква.

По същия начин, както за CREATE VIEW v_ стандартното именуване за съхранените процедури е

CREATE PROCEDURE usp_ което означава User Stored Procedure. В примера в началото съм го изписала нарочно грешно, за да демонстрирам редакция по-надолу.

Limit to 1000 rows

```
1   USE geography;
2
3   DELIMITER $$
4
5   CREATE PROCEDURE GetCountries()
6   BEGIN
7       SELECT
8           country_name,
9           capital,
10          population
11      FROM
12          countries
13      ORDER BY population DESC, country_name ASC;
14  END $$
15  DELIMITER ;
```

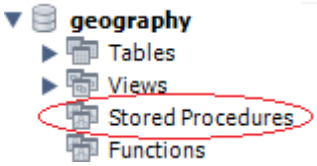
Tab - за форматиране

- Това намалява мрежовия **трафик** към локалния сървър;
- Много по-лесно се организира **логиката** – кодът се пази, а само когато ни потрябва, ние извикваме процедурата. Това съответства на правилото в програмирането за неповтарящ се код DRY – Don't repeat yourself. Винаги отделяме повтарящият се код другаде и само извикваме запазения метод.
- Повишава се **производителността**, т.к. кодът не се компилира всеки път наново – той се пази. Обработват се само подадените му входни данни.
- Последно – това повишава и **сигурността**, когато се използват различни приложения Applications.

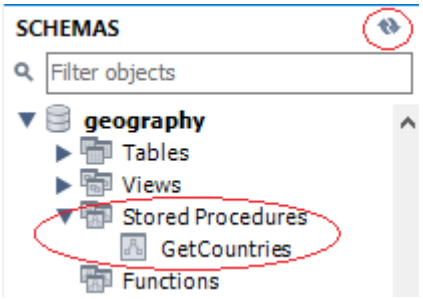
63	03:35:29	USE geography	0 row(s) affected
64	03:35:49	CREATE PROCEDURE GetCountries() B...	0 row(s) affected

Макар, че видимо нищо не се случва както при стандартните заявки – резултатът не се показва на екрана, ние се уверяваме, че кодът е изпълнен – имаме зелен тик за CREATE PROCEDURE, но по същия начин както в CREATE VIEW, изходът не се екранизира и изписва 0 row(s) affected. Това не бива да ни притеснява.

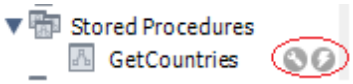
Поглеждаме вляво резултата

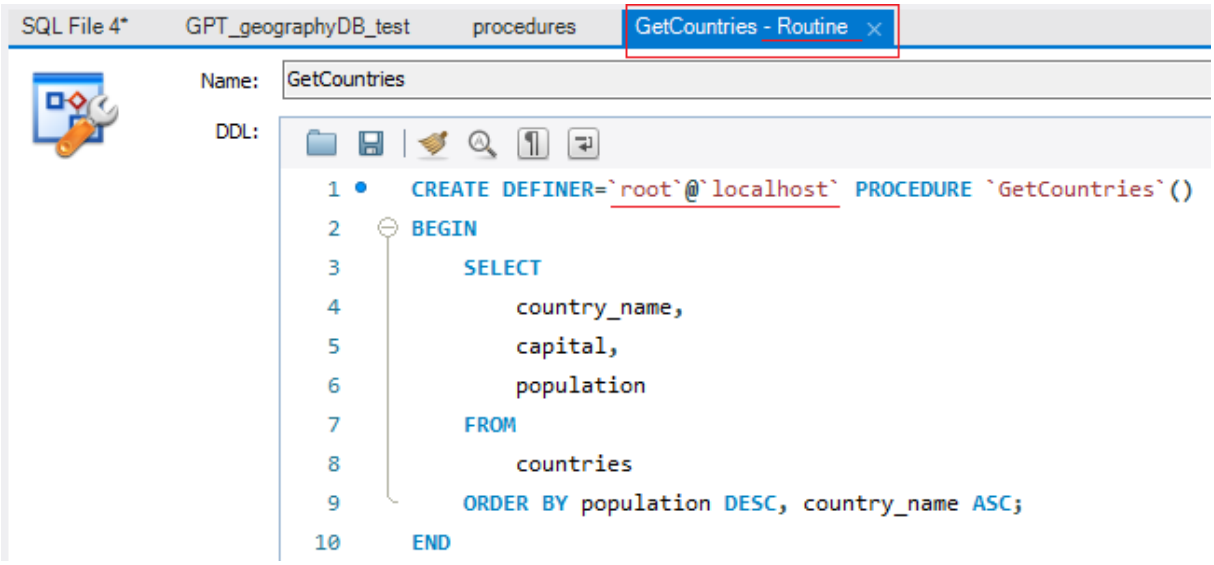


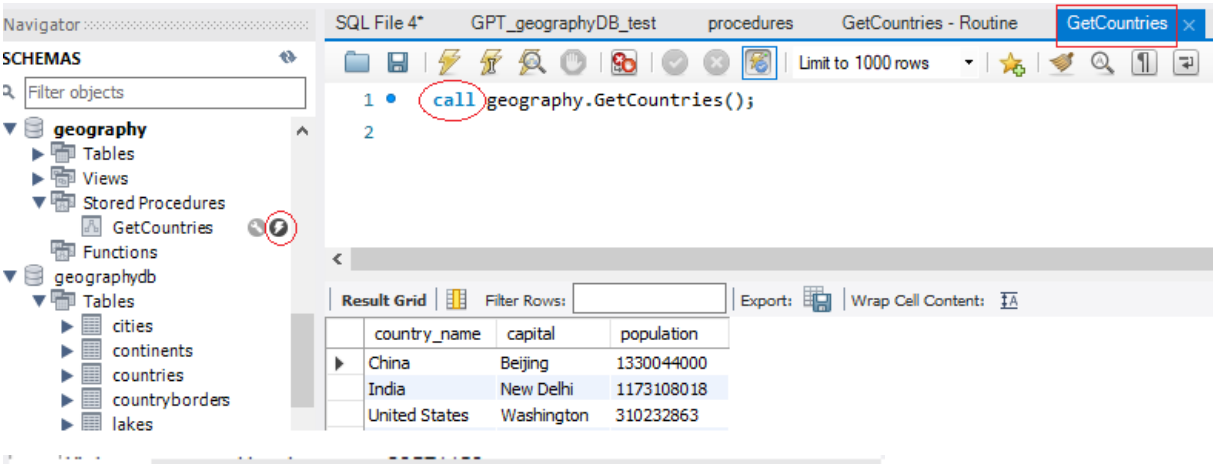
преди / след Refresh:

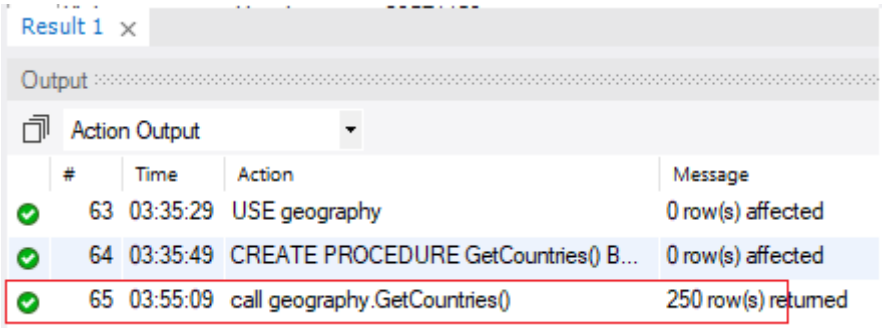


Готовата процедура има само 2 възможности – настройки и изпълнение:

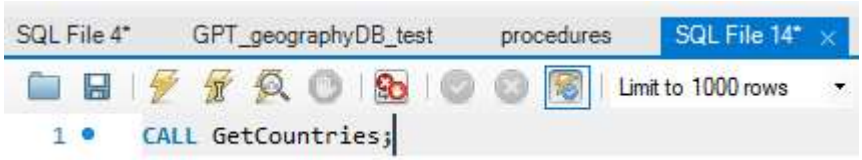






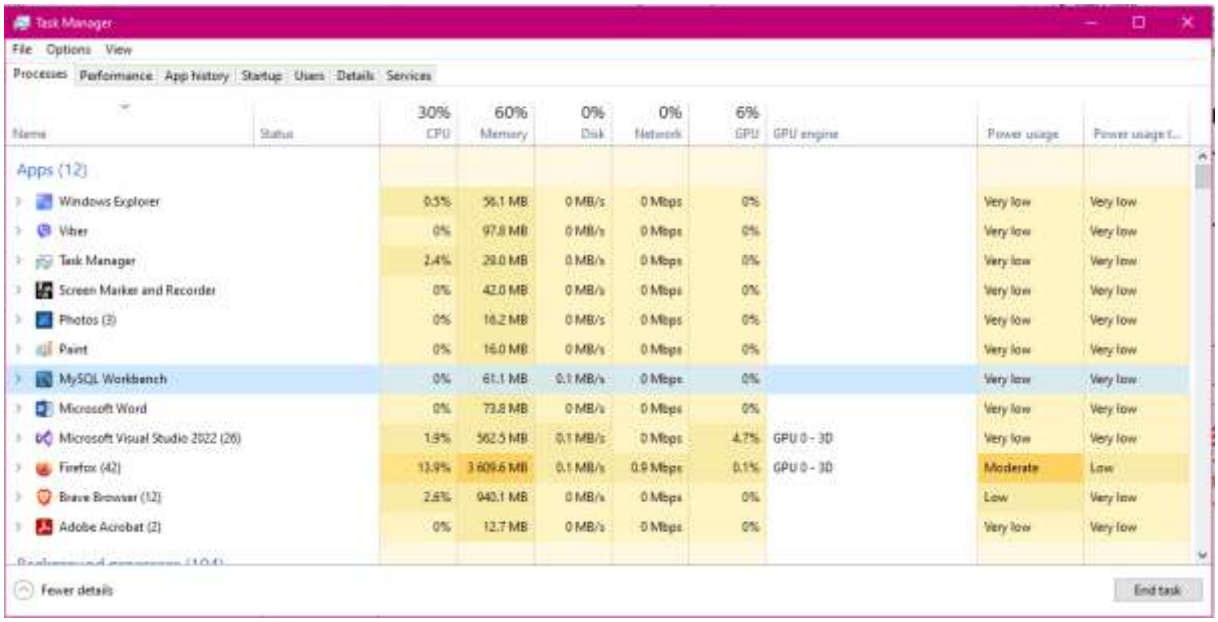


За да извикаме вече съхранена процедура, просто използваме CALL. Желателно е в нов подпрозорец, за да се отдели кода!!! Това е като защитна мярка, т.к. иначе хвърля грешки – т.е. целта е наистина процедурата да е отделена от кода, съхранена другаде, а ние, когато пишем, просто да я извикваме като парченце (метод) в съвсем отделен актуален код.



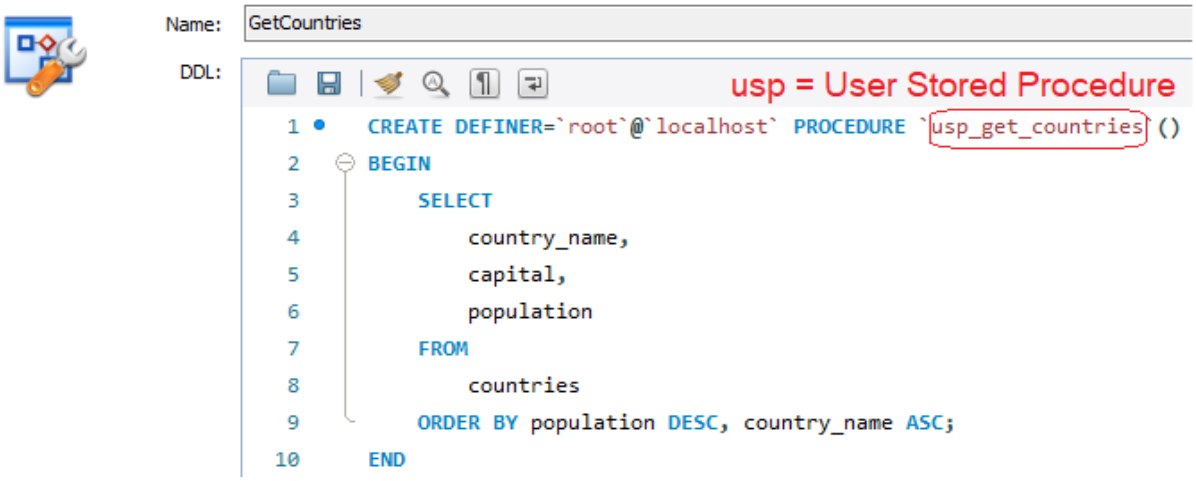
Недостатъци:

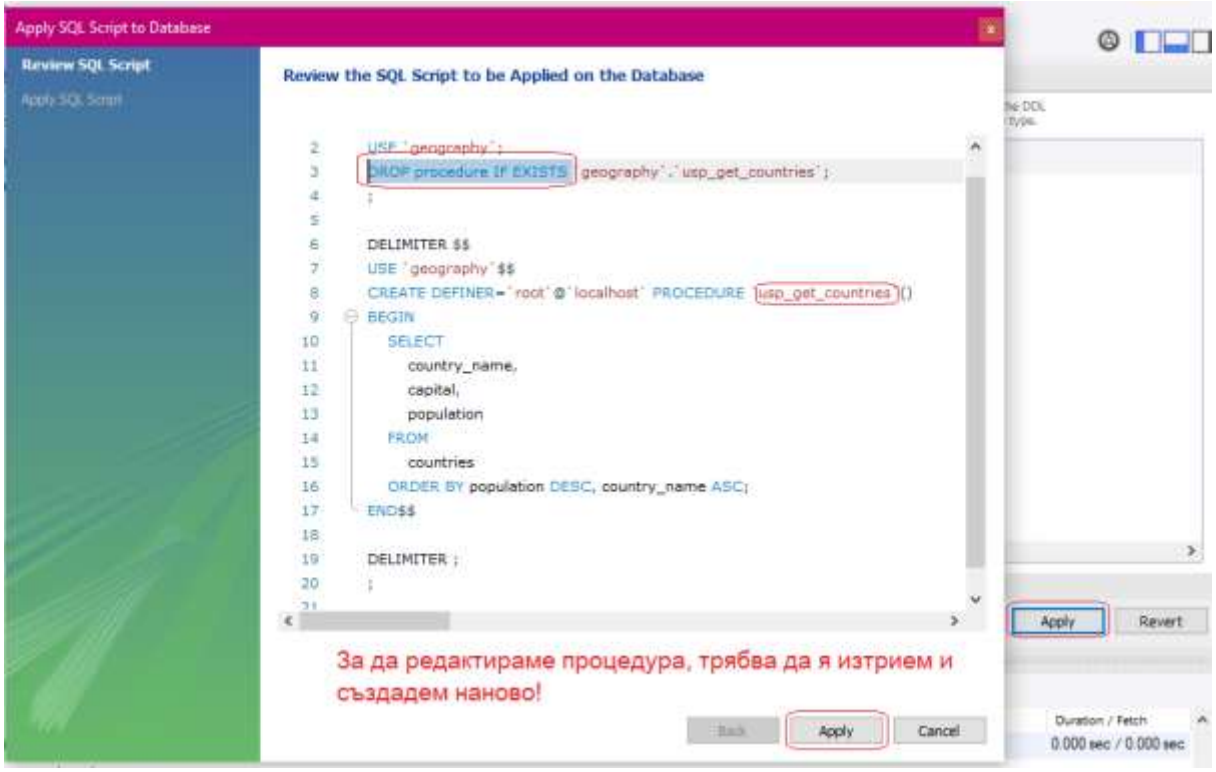
* SQL не е създаден да съхранява много процедури – те ще натоварят паметта и разходите на процесора.



Редакциите на вече съхранени процедури са голямо и трудно предизвикателство.

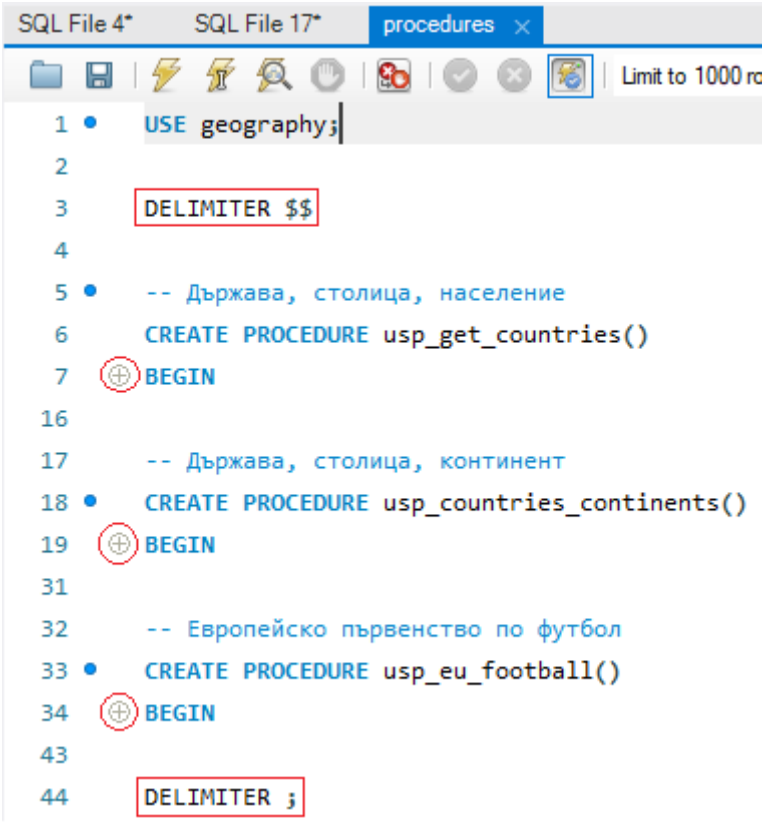
Нека проверим колко трудно се редактира процедура като сменим името й с по-правилното по конвенцията snake_case за SQL –





При процедурите няма ALTER както при таблиците – да се редактират настройките, но запазят данните, а директно DROP -> CREATE.

Можем да създадем няколко процедури в един файл, но уточнихме, че не е желателно паметта да се претрупва с процедури.



Втората задача е лесна – с JOIN, като, понеже имената на съединените таблици започват с една и съща буква, първата таблица ще бъде a, втората – b.

```
-- Държава, столица, континент
CREATE PROCEDURE usp_countries_continents()
BEGIN
    SELECT
        country_name AS Country,
        capital AS Capital,
        continent_name AS Continent
    FROM
        countries a
    JOIN
    continents b
    ON a.continent_code = b.continent_code
    ORDER BY population DESC, country_name ASC;
END $$
```

Третата задача за Европейското първенство по футбол е интересна. Трябва всяка държава да играе с всяка (Декартово произведение CROSS JOIN) И да се уверим, че няма държава, която играе в себе си, а накрая подредбата да бъде на случаен RANDOM() принцип.

```
-- Европейско първенство по футбол
CREATE PROCEDURE usp_eu_football()
BEGIN
    SELECT a.capital AS Place,
           a.country_name AS "Player 1 (Host)", " " AS "Host", " " AS "Guest",
           b.country_name AS "Player 2 (Guest)"
    FROM countries a
    CROSS JOIN countries b
    WHERE a.continent_code = "EU" AND b.continent_code = "EU" AND a.country_code <> b.country_code
    ORDER BY RAND();
END $$
```

!= в SQL

Изпълняваме:

```
CALL usp_eu_football();
```

Примерен изход:

Place	Player 1 (Host)	Host	Guest	Player 2 (Guest)
Stockholm	Sweden			Bosnia and Herzegovina
London	United Kingdom			Serbia
Moscow	Russia			Luxembourg
Nicosia	Cyprus			Montenegro
Chisinau	Moldova			Norway
Pristina	Kosovo			Lithuania
Kyiv	Ukraine			Montenegro
St Peter Port	Guernsey			Belarus
Nicosia	Cyprus			Czech Republic

Макар, че имаме CROSS JOIN, който трябва да изведе резултат 250 x 250 = 62500 мача, заради ограниченията, изходът в задачата е 2750 реда/мача.

Action Output

#	Time	Action	Message
1	10:07:19	CALL usp_eu_football()	2756 row(s) returned

Момчетата да мислят как да подобрят условията.

В скобите могат да се подадат входни данни с определен тип, да се върне резултат. По подобен начин се правят функции.

- Tables
- Views
- Stored Procedures
- Functions

ЗАКЛЮЧИТЕЛНИ ДУМИ

Много полезен линк за TIMESTAMP – времеви печат: https://www.mysqltutorial.org/mysql-basics/understanding-mysql-timestamp/

Функции: https://www.mysqltutorial.org/mysql-string-functions/

Целият туториъл е много полезен – дори може да се изтегли примерна БД.

Примерни функции – това вече си е програмиране с входен и изходен резултат и if-else:

```
DELIMITER $$

CREATE FUNCTION udf_project_weeks(start_date DATETIME, end_date DATETIME)
RETURNS INT
BEGIN
    DECLARE project_weeks INT;
        IF(end_date IS NULL) THEN
        SET end_date := NOW();
        END IF;
    SET project_weeks := DATEDIFF(DATE(end_date), DATE(start_date)) / 7;
    RETURN project_weeks;
END $$
```

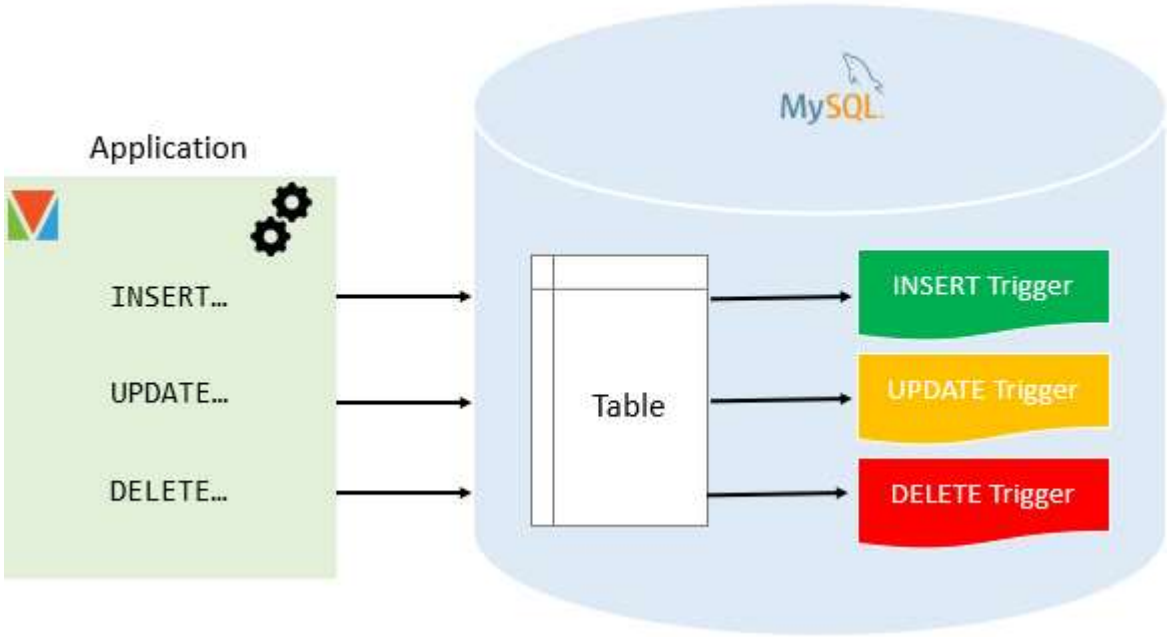
……

```
CREATE FUNCTION udf_get_salary_level(salary DECIMAL(19,4))
RETURNS VARCHAR(10)
BEGIN
    DECLARE salary_level VARCHAR(10);
    IF (salary < 30000) THEN
        SET salary_level := 'Low';
        ELSEIF(salary >= 30000 AND salary <= 50000) THEN
        SET salary_level := 'Average';
        ELSE
        SET salary_level := 'High';
        END IF;
    RETURN salary_level;
END $$
```

Транзакции – изпълняват се в цялост или всичко, или нищо, без загуба. Пример са банковите преводи. Промените се съхраняват едва след изпълнението на COMMIT. По всяко време всички промени могат да се отменят с ROLLBACK https://www.mysqltutorial.org/mysql-stored-procedure/mysql-transactions/

```
DELIMITER $$

START TRANSACTION
UPDATE accounts SET balance = balance withdraw_amount
WHERE id = account
IF ROW_COUNT() <> 1 THEN Affected rows are different than one
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid account';
ROLLBACK;
ELSE
COMMIT;
END IF $$
```

Тригери - извикват се в случай на дадено събитие, а не изрично с CALL. Те се прикрепят към таблицата и се изпълняват, когато дадена заявка се изпълнява върху съдържанието на таблицата, например при добавяне на нов ред. https://www.mysqltutorial.org/mysql-triggers/

```
DELIMITER $$

CREATE TRIGGER tr_delete_records
AFTER DELETE
ON employees_projects
FOR EACH ROW
BEGIN
    INSERT INTO employees_projects_history
    (employee_id, project_id)
    VALUES(old.employee_id, old.project_id);
END $$
```